TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

CẨM NANG LUYỆN NÓI & PHẢN BIỆN CÁ NHÂN
ĐÀO TRÍ KHANG (1923040156)

Vai trò: Mobile Software Engineer – Phụ trách Android Native Java | Thứ tự thuyết trình: Người số 2 (Trình bày Mobile & Threading) | Thời lượng mục tiêu: 3 phút 15 giây (Khóa cứng thời gian)
Dự án: Hệ thống Quản lý Hoạt động Hội nhập – VISIONARY

📌 **CHIẾN THUẬT DÀNH CHO TRÍ KHANG:** Khang phụ trách phần ứng dụng di động Android Native Java. Trọng tâm giảng viên luôn hỏi xoay quanh: (1) Tại sao không thể gọi mạng trên Main Thread; (2) Cơ chế Retrofit enqueue & Handler/Looper đẩy kết quả về UI; (3) Địa chỉ 10.0.2.2 trên Android Emulator; (4) Khai báo usesCleartextTraffic và quyền INTERNET.

I. PHÂN BỔ SLIDE VÀ THỜI GIAN (3 PHÚT 15 GIÂY)

Slide	Tiêu đề Slide	Nội dung trọng tâm	Thời lượng
Slide 5	Ứng dụng Android Native Java	Kiến trúc Android Java, Retrofit 2, Gson, RecyclerView, các điều khiển và snippet code	1 phút 00 giây
Slide 6	Android Gọi API & Cơ Chế Luồng	Phân biệt Network Thread và UI Thread, xử lý bất đồng bộ, Base URL 10.0.2.2 & Token	1 phút 00 giây
Slide 7	Demo Trực Tiếp Android App	Thao tác trên Android Emulator: Đăng nhập, tải sự kiện, đăng ký tham gia nhận thông báo	1 phút 15 giây

II. LỜI THUYẾT TRÌNH TỪNG CÂU (HỌC THUỘC LÒNG & NÓI TỰ NHIÊN)

1. Khi ở Slide 5 (Android Native Java) – 1 phút 00 giây:

"Kính chào Thầy và các bạn, em là Đào Trí Khang. Tiếp nối phần trình bày của bạn Đức, em xin giới thiệu ứng dụng di động VISIONARY dành cho sinh viên, được xây dựng hoàn toàn bằng Android Native với ngôn ngữ Java.

Ứng dụng sử dụng kiến trúc phân tầng rõ ràng: Giao diện dùng RecyclerView hiển thị danh sách thẻ sự kiện, mô hình dữ liệu ánh xạ tự động qua thư viện Gson, và tầng kết nối mạng được quản lý tập trung bằng Retrofit 2 kết hợp OkHttp client.

Trên màn hình là đoạn mã khai báo Retrofit interface trong tệp VisionaryApiService.java. Nhóm khai báo endpoint @GET('api/events') trả về kiểu Call bọc ApiResponse chứa PagedResponse của EventItem. Nhờ khai báo đúng cấu trúc Generic phân trang này, Retrofit tự động giải mã JSON phức tạp từ server mà không gây lỗi ép kiểu."

2. Khi ở Slide 6 (Android Gọi API & Luồng Xử Lý) – 1 phút 00 giây:

"Thưa Thầy, một trong những thách thức lớn nhất trên Android là quản lý luồng thực thi (threading). Hệ điều hành Android cấm tuyệt đối việc thực hiện cuộc gọi mạng trên Main Thread; nếu vi phạm sẽ lập tức ném ra ngoại lệ NetworkOnMainThreadException làm ứng dụng tắt ngay.

Để giải quyết, nhóm sử dụng phương thức .enqueue() bất đồng bộ của Retrofit. Yêu cầu HTTP được đẩy sang Background Thread chạy ngầm. Khi server phản hồi, Retrofit tự động điều phối dữ liệu trở lại Main UI Thread thông qua Looper và Handler, đảm bảo giao diện cập nhật an toàn mà không làm đơ giật ứng dụng.

Đồng thời, khi chạy thử nghiệm trên máy ảo Android Emulator, địa chỉ IP máy chủ không thể dùng 'localhost' mà bắt buộc phải trỏ vào 10.0.2.2 – đây là địa chỉ loopback đặc biệt trỏ về máy tính thật. Mọi yêu cầu đều được tự động gắn mã JWT Bearer Token qua AuthInterceptor."

3. Khi ở Slide 7 (Demo Android Trực Tiếp) – 1 phút 15 giây:

"Sau đây, em xin phép demo trực tiếp ứng dụng Android đang chạy trên máy ảo Android Studio.

[KHANG THAO TÁC LIVE TRÊN ANDROID EMULATOR]:
- Bước 1: Mở app, giao diện tải danh sách sự kiện từ Web API qua lệnh GET /api/events. Các sự kiện hiển thị mượt mà trên RecyclerView.
- Bước 2: Chọn sự kiện 'Hội thảo Kỹ năng Hội nhập Quốc tế', màn hình chuyển sang EventDetailActivity.
- Bước 3: Nhấn nút 'Đăng ký tham gia' (btnDangKy). Ứng dụng gửi yêu cầu POST /api/events/{id}/register kèm JWT Token của sinh viên.
- Bước 4: Server phản hồi HTTP 200 thành công, giao diện hiển thị Dialog chúc mừng và mã QR điểm danh.

Dữ liệu đăng ký này đã lập tức được lưu vào cơ sở dữ liệu chung và sẽ được anh Đạt kiểm tra ngay trên WinForms ở phần sau.

Sau đây, em xin chuyển lời sang bạn Hồ Đoàng Nguyên Khang để phân tích chuyên sâu về Web API, tài liệu Swagger và hợp đồng dữ liệu. Xin mời Khang!"

III. MÃ NGUỒN CẦN MỞ VÀ 3 ĐIỂM NÓNG GIẢNG VIÊN DỄ TRỎ VÀO

Tệp mã nguồn Khang phụ trách mở khi giảng viên kiểm tra mã Android:

📁 File API Service:
mobile/VisionaryAndroid/app/src/main/java/com/visionary/studentconnect/api/VisionaryApiService.java

📁 File UI Fragment:

mobile/VisionaryAndroid/app/src/main/java/com/visionary/studentconnect/ui/events/EventsFragment.java

📁 File Interceptor: mobile/VisionaryAndroid/app/src/main/java/com/visionary/studentconnect/api/AuthInterceptor.java

📘 Mã nguồn minh chứng: Đoạn mã Retrofit enqueue trong EventsFragment.java

```
// VisionaryApiService.java
@GET("api/events")
Call<ApiResponse<PagedResponse<EventItem>>> getEvents(
    @Query("pageNumber") int page,
    @Query("pageSize") int pageSize,
    @Query("status") String status
);

// EventsFragment.java - Gọi bất đồng bộ qua enqueue
apiService.getEvents(1, 10, null).enqueue(new Callback<ApiResponse<PagedResponse<EventItem>>>()
{
    @Override
    public void onResponse(Call<ApiResponse<PagedResponse<EventItem>>> call, Response<...>
response) {
        if (response.isSuccessful() && response.body() != null) {
            List<EventItem> items = response.body().getData().getItems();
            eventAdapter.setEvents(items); // Cập nhật trên UI Thread
        }
    }
    @Override
    public void onFailure(Call<...> call, Throwable t) {
        showErrorMessage("Lỗi kết nối máy chủ: " + t.getMessage());
    }
});
```

👉 Điểm 1: Phương thức '.enqueue(new Callback<...>)':

Giảng viên hỏi: Tại sao lại gọi .enqueue() mà không gọi .execute()?
-> Khang trả lời: 'Dạ thưa Thầy, .execute() là gọi đồng bộ (Synchronous), nó sẽ chặn Main Thread của Android và gây ra lỗi NetworkOnMainThreadException ngay lập tức. Phương thức .enqueue() là gọi bất đồng bộ (Asynchronous), Retrofit tự tạo Worker Thread chạy ngầm và trả kết quả về Main Thread qua Callback ạ.'

👉 Điểm 2: Địa chỉ 10.0.2.2 trong cấu hình API:

Giảng viên hỏi: Tại sao trong tệp cấu hình Android lại viết http://10.0.2.2:5230 thay vì localhost?
-> Khang trả lời: 'Dạ thưa Thầy, đối với máy ảo Android Emulator, localhost hay 127.0.0.1 đại diện cho chính bản thân hệ điều hành Android ảo đó. Để máy ảo truy cập ra máy tính chủ (Host Machine) nơi Backend API đang lắng nghe, Android dành riêng địa chỉ bí danh 10.0.2.2 làm cổng gateway kết nối về host ạ.'

👉 Điểm 3: Khai báo usesCleartextTraffic trong AndroidManifest.xml:

Giảng viên hỏi: Dòng android:usesCleartextTraffic="true" để làm gì? Xóa đi thì sao?
-> Khang trả lời: 'Dạ thưa Thầy, từ Android 9 (API 28) trở lên, Google mặc định chặn toàn bộ kết nối HTTP không mã hóa (Cleartext Traffic). Dòng này cho phép ứng dụng gọi HTTP trong môi trường phát triển local. Nếu xóa đi và gọi HTTP, ứng dụng sẽ bị ném lỗi java.io.IOException: Cleartext HTTP traffic not permitted và không thể tải dữ liệu.'

IV. BỘ CÂU HỎI PHẢN BIỆN CHUYÊN SÂU ANDROID (TRẢ LỜI 20-30 GIÂY)

❓ Câu 1: Lỗi NetworkOnMainThreadException khác gì với CalledFromWrongThreadException?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, NetworkOnMainThreadException xảy ra khi ta cố tình thực hiện thao tác mạng (I/O) trên luồng giao diện chính. Còn CalledFromWrongThreadException xảy ra theo chiều ngược lại: khi ta đang ở trong luồng nền (Background Thread) mà cố tình sửa đổi trực tiếp các View của giao diện mà không đẩy qua Handler.

❓ Câu 2: Token đăng nhập trên Android được lưu ở đâu? Có mất khi tắt ứng dụng không?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, trên Android nhóm lưu Token vào SharedPreferences của ứng dụng. Dữ liệu này được lưu xuống bộ nhớ flash của máy nên khi người dùng thoát ứng dụng hoàn toàn hoặc khởi động lại điện thoại, Token vẫn còn nguyên và tự động nạp lại.

❓ Câu 3: Làm thế nào Android gắn Token vào mỗi cuộc gọi API mà không cần truyền thủ công ở từng hàm?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, nhóm sử dụng AuthInterceptor trong OkHttp. Trước khi bất kỳ request nào được gửi ra mạng, Interceptor sẽ chặn lại, lấy Token từ SharedPreferences và tự động thêm vào header 'Authorization: Bearer <token>', giúp toàn bộ các hàm gọi API khác hoàn toàn trong suốt.

❓ Câu 4: Phân biệt lỗi mất mạng (Không có internet) và lỗi máy chủ từ chối kết nối (Server Down)?

🗣️ Khang trả lời (20-30s): Dạ thưa Thầy, nếu mất mạng thì Throwable trả về UnknownHostException (không phân giải được tên miền). Còn nếu có mạng nhưng Server không bật hoặc sai cổng thì Throwable trả về ConnectException (Connection refused). Cả hai đều được hứng ở nhánh onFailure() của Retrofit.

V. CHECKLIST TỰ LUYỆN TRƯỚC BUỔI BẢO VỆ

- [ ] Mở sẵn Android Studio và khởi động máy ảo Emulator trước giờ thuyết trình 15 phút.
- [ ] Cài sẵn file app-debug.apk đã biên dịch (6.4 MB) vào máy ảo, không compile lại tại chỗ.
- [ ] Kiểm tra máy ảo kết nối được internet và API.
- [ ] Luyện tập nói Slide 5, 6, 7 bấm giờ chuẩn dưới 3 phút 15 giây.
- [ ] Nhớ câu kết chuyển giao: 'Sau đây em xin chuyển lời sang bạn Hồ Đoàng Nguyên Khang...'